MODULE 07 · DOMAIN 1 - SDLC AUTOMATION

Containers and Serverless Deployment

ECS rolling updates and native blue/green, CodeDeploy for ECS, Lambda traffic shifting with SAM, Elastic Beanstalk policies, and EKS.

DOMAIN 1 of 6 EXAM WEIGHT 22% MODULE 7 of 34

admin@sthithapragna.com

On every platform in this module, a deployment is a decision about **where traffic goes**. The platform only changes what traffic is pointed at.

Who moves the traffic, on each platform

PLATFORM	ROLLING	BLUE/GREEN, CANARY, LINEAR
Amazon ECS	ECS itself	ECS natively (2025), or CodeDeploy
AWS Lambda	Not applicable	CodeDeploy through an alias; SAM configures it
Elastic Beanstalk	Beanstalk policies	Immutable, traffic splitting, or a URL swap
Amazon EKS	Kubernetes Deployments	Progressive delivery tools in the cluster

ON THE EXAM

CodeDeploy is optional for ECS now, but it is still the engine behind Lambda traffic shifting, including through SAM.

ECS rolling updates: two numbers

- **minimumHealthyPercent**: the floor of running tasks, as a percentage of desired count.
- **maximumPercent**: the ceiling of running tasks during the deployment.
- 100 and 200: never below full strength, but needs room for twice the tasks.
- 50 and 100: no extra room needed, but runs at half strength during the update.

> ON THE EXAM
>
> 'Unable to place a task' during deployments on a full cluster: lower minimumHealthyPercent, or add capacity.

Two ways a rolling update notices trouble

DEPLOYMENT CIRCUIT BREAKER

Tasks that will not run

- Tasks fail to start, or fail health checks
- Counts failures against a threshold
- With rollback on: returns to the last completed deployment
- Blind to a task that runs but misbehaves

DEPLOYMENT ALARMS

Tasks that run badly

- CloudWatch alarms named on the service
- Error rate, latency, any metric you choose
- With rollback on: returns to the last completed deployment
- Catches the bug the circuit breaker cannot see

THE TIEBREAKER

Tasks crash-loop: circuit breaker. Tasks healthy but errors rise: deployment alarms. Many services want both.

ECS native blue/green, stage by stage

01

Scale up green

New tasks start. Blue still takes 100% of production.

02

Test traffic

A test listener sends test requests to green.

03

Production shift

All at once, linear steps, or a canary slice.

04

Bake

Both run. Alarms watch. Rollback is instant.

05

Clean up

Blue scales to zero. Green is production.

ON THE EXAM

Rollback before clean-up just moves traffic back to blue tasks that are still running. No new tasks start.

Lifecycle hooks: where you can intervene

STAGE	STATE AT THAT MOMENT	TYPICAL HOOK
PRE_SCALE_UP	Green not started	Check dependencies are ready
POST_SCALE_UP	Green running, no traffic	Smoke-test green directly
POST_TEST_TRAFFIC_SHIFT	Green has all test traffic	Run the test suite via the test listener
PRE_PRODUCTION_TRAFFIC_SHIFT	Before each production step	Check metrics before the next slice
POST_PRODUCTION_TRAFFIC_SHIFT	Green has all production traffic	Final validation before bake

ON THE EXAM

A Lambda hook returns SUCCEEDED, FAILED to roll back, or IN_PROGRESS to be asked again in 30 seconds. PAUSE waits for you.

Linear, canary and bake time on ECS

- **Linear**: a step percentage and a bake time between steps. **Canary**: one slice, a bake, then the rest.
- **Bake time**, up to 24 hours, keeps blue running after the final shift for instant rollback.
- Works with an Application Load Balancer, a Network Load Balancer, or ECS Service Connect.
- The deployment controller can now be changed after the service is created.

> ON THE EXAM
>
> AWS now recommends ECS-native strategies for new deployments. CodeDeploy for ECS remains supported and examinable.

ECS native, or CodeDeploy for ECS?

ECS NATIVE

Built into the service

- No CodeDeploy application or group
- ALB, NLB or Service Connect
- Hooks at more stages, plus PAUSE
- Recommended for new services

CODEDEPLOY

The long-established route

- Deployment controller CODE_DEPLOY
- Appspec names the task definition
- Predefined ECS canary and linear configs
- The exam guide's world; still valid

THE RULE

If an option uses CodeDeploy blue/green for ECS, it is still a correct design. Choose between options on the qualifier.

A CodeDeploy appspec for ECS

```
1  version: 0.0
2  Resources:
3    - TargetService:
4        Type: AWS::ECS::Service
5        Properties:
6          TaskDefinition: "arn:aws:ecs:...:task-definition/web:42"
7          LoadBalancerInfo:
8            ContainerName: "web"
9            ContainerPort: 8080
10 Hooks:
11   - AfterAllowTestTraffic: "arn:aws:lambda:...:function:run-tests"
```

6 The new task definition revision to deploy.

11 Hooks here are Lambda functions, not scripts on an instance.

ON THE EXAM

ECS predefined configs: ECSAllAtOnce, ECSCanary10Percent5Minutes or 15Minutes, ECSLinear10PercentEvery1Minutes or 3Minutes.

Lambda: versions, aliases and weights

- A **version** is an immutable snapshot of code and configuration.
- An **alias** is a named pointer to a version, and callers invoke the alias.
- An alias can **split** invocations between two versions by weight.
- CodeDeploy moves that weight on a schedule, and rolls it back on an alarm.

> ON THE EXAM
>
> Callers invoking $LATEST or a version number directly bypass the alias, and so bypass traffic shifting.

SAM does the wiring

```
1  OrdersFunction:
2    Type: AWS::Serverless::Function
3    Properties:
4      Handler: app.handler
5      Runtime: python3.13
6      AutoPublishAlias: live
7      DeploymentPreference:
8        Type: Canary10Percent10Minutes
9        Alarms: [!Ref OrdersErrorAlarm]
10       Hooks:
11         PreTraffic: !Ref PreTrafficCheck
```

6 Publish a new version on every change and point 'live' at it.

8-9 Shift 10% for ten minutes; roll back if the alarm fires.

ON THE EXAM

Those two properties create the CodeDeploy application, group, and alias shifting. No CodeDeploy resources to write.

Other ways to shift Lambda traffic

- The **CodePipeline Lambda deploy action** (May 2025, V2 only) supports canary and linear with custom values.
- It rolls back automatically when named CloudWatch alarms fire during the shift.
- Predefined CodeDeploy configs: LambdaCanary10Percent 5, 10, 15, 30 minutes; LambdaLinear10Percent every 1, 2, 3, 10 minutes.
- Lambda hooks must call **PutLifecycleEventHookExecutionStatus**, or the deployment waits, then fails.

> ON THE EXAM
>
> Same lesson as a pipeline's Lambda action: custom code in a deployment must report its result back.

Elastic Beanstalk deployment policies

POLICY	CAPACITY DURING	IF IT FAILS
All at once	Outage	Redeploy the old version
Rolling	Reduced by one batch	Mixed versions; redeploy
Rolling with additional batch	Full	Mixed versions; redeploy
Immutable	Double, in a temporary group	Terminate the new group: fast
Traffic splitting	Double, with a canary share	Shift traffic back: fast

ON THE EXAM

Full capacity, cheapest: rolling with additional batch. Fast, clean rollback: immutable or traffic splitting.

In-environment policies, or a URL swap?

IMMUTABLE OR TRAFFIC SPLITTING

Inside one environment

- Same load balancer, same DNS name
- No client DNS caching problems
- For application version changes
- Rollback terminates new instances

BLUE/GREEN URL SWAP

Two whole environments

- Clone, deploy, then swap environment URLs
- A DNS change: clients may cache the old one
- For platform branch or major config changes
- Rollback is another swap

THE TIEBREAKER

Changing platform branch, such as leaving retired Amazon Linux 2: clone and swap. Routine releases: immutable.

Configuring Beanstalk from source

- **.ebextensions** config files set options and create resources during deployment.
- **.platform/hooks** scripts run at defined points on Amazon Linux platforms.
- **Managed platform updates** apply patches in a weekly maintenance window.
- A database created inside the environment is deleted with it. Keep production databases outside.

> ON THE EXAM
>
> Blue/green needs a database both environments share, which is another reason it must live outside the environment.

Amazon EKS: Kubernetes does the rolling

- A Deployment's rolling update is tuned by **maxSurge** and **maxUnavailable**.
- A **readiness probe** decides when a pod may receive traffic.
- A **liveness probe** decides when a stuck container should be restarted.
- Canary and blue/green on EKS usually come from progressive delivery tools running in the cluster.

> ON THE EXAM
>
> New pods get errors while warming up: a readiness probe that passes only when the pod can really serve.

> An ECS service behind an Application Load Balancer uses rolling updates with the deployment circuit breaker and rollback enabled.
>
> A new release starts cleanly and every task passes its health checks, but the ALB's 5xx error rate doubles. The deployment completes and nothing rolls back.

What should the DevOps engineer add so that such releases roll back automatically?

A. A CloudWatch alarm on the target group's 5xx rate, named in the service's deployment alarms with rollback enabled

B. A higher circuit breaker failure threshold

C. A lower minimumHealthyPercent on the service

D. ECS Exec on the tasks, so engineers can investigate faster

◆ PAUSE AND DECIDE

What should the DevOps engineer add so that such releases roll back automatically?

A Deployment alarm on 5xx

Watches what the circuit breaker cannot: tasks that run healthily but return errors.

B Higher circuit breaker threshold

No task ever failed, so no threshold, high or low, would be reached.

C Lower minimumHealthyPercent

Changes how many tasks run during the update, not whether bad ones are detected.

D ECS Exec

Helps a human investigate. The requirement was automatic rollback.

THE SIGNAL

Healthy tasks, rising errors, no rollback: the circuit breaker only sees tasks that fail. Add deployment alarms.

> An ECS service runs 4 tasks on an EC2 capacity provider that has no spare room, and the company will not add instances. The service uses minimumHealthyPercent 100 and maximumPercent 200.
>
> Every deployment stalls with an 'unable to place a task' event. Briefly running at half capacity during deployments is acceptable.

What should the DevOps engineer change?

A. Set maximumPercent to 300

B. Set minimumHealthyPercent to 50 and maximumPercent to 100

C. Switch the service to a blue/green deployment strategy

D. Increase the service's desired count to 8

◆ PAUSE AND DECIDE

What should the DevOps engineer change?

A maximumPercent 300

Allows even more extra tasks, on a cluster with no room for one.

B 50 and 100

Stops two old tasks first, then starts two new ones in their place. No extra room needed.

C Blue/green

Needs a full second set of tasks at once: the most extra room of all.

D Desired count 8

Needs double the capacity permanently. The opposite of the constraint.

THE SIGNAL

No spare capacity, reduced capacity acceptable: lower minimumHealthyPercent and keep maximumPercent at 100.

A team wants to run its automated API test suite against each new ECS release on production infrastructure before any customer request reaches it. If the tests fail, the release must roll back automatically.

The team does not want to create or manage CodeDeploy resources, and wants the least operational overhead.

Which solution meets these requirements?

A. ECS native blue/green with a test listener rule, and a Lambda lifecycle hook at POST_TEST_TRAFFIC_SHIFT that runs the tests

B. A rolling update with the circuit breaker and rollback enabled

C. A separate staging service, with a manual approval before production

D. A CodeDeploy canary configuration that sends 10% of production traffic for 5 minutes

◆ PAUSE AND DECIDE

Which solution meets these requirements?

A Native blue/green, test listener, hook

Green is tested on production infrastructure through the test listener, with no customers on it. A FAILED hook rolls back.

B Rolling with circuit breaker

No test stage: customers reach new tasks as soon as they are healthy.

C Staging and approval

Not production infrastructure, and not automatic.

D CodeDeploy canary

Tests on real customers, and uses the CodeDeploy resources the team wants to avoid.

THE SIGNAL

'Before any customer request' plus 'on production infrastructure' is a test listener. 'No CodeDeploy' makes it native.

> An API runs on Amazon API Gateway and a Lambda function, deployed with AWS SAM. The team wants each release to receive 10% of requests for ten minutes before taking all traffic.
>
> If the function's error alarm fires during that time, the release must roll back automatically. The team wants the least effort.

What should the team add to the SAM template?

A. A script that publishes a version and updates the alias weights after ten minutes

B. AutoPublishAlias, and a DeploymentPreference of Canary10Percent10Minutes naming the alarm

C. An API Gateway canary release on the stage, with 10% of traffic

D. Two separate functions behind Route 53 weighted records

◆ PAUSE AND DECIDE

What should the team add to the SAM template?

A **Custom script**

Rebuilds by hand the shifting and rollback that CodeDeploy provides.

B **AutoPublishAlias plus DeploymentPreference**

SAM creates the CodeDeploy wiring. Ten percent, ten minutes, alarm rollback: all declared.

C **API Gateway canary**

Splits at the API stage, but promotion and rollback on the alarm are not automatic.

D **Route 53 weighted**

DNS caching makes percentages imprecise, and nothing rolls back.

THE SIGNAL

SAM plus gradual Lambda release plus alarm rollback: AutoPublishAlias and DeploymentPreference, every time.

A Lambda function is deployed through CodeDeploy with a BeforeAllowTraffic hook. The hook is itself a Lambda function that invokes the new version and checks the response.

The hook's logs show its checks pass within seconds, yet every deployment stays in progress for a long time and then fails. Traffic never shifts.

What is the most likely cause?

A The hook function's timeout is too short

B The alias is pointing at $LATEST

C The hook never calls PutLifecycleEventHookExecutionStatus

D The deployment should use a linear configuration instead

◆ PAUSE AND DECIDE

What is the most likely cause?

A Hook timeout too short

The checks finished in seconds. A short timeout would show in the logs as a timeout.

B Alias on $LATEST

Would cause other errors. It does not explain a successful hook followed by a hang.

C No PutLifecycleEventHookExecutionStatus

The hook succeeds but never tells CodeDeploy, which waits, then fails.

D Use linear

The configuration shapes the shift. The shift never started.

THE SIGNAL

Hook succeeds, deployment hangs, then fails: the hook never reported its status. Same lesson as module 2.

A company deploys a web application to Elastic Beanstalk. Deployments must keep full capacity, and a failed release must be removed quickly without affecting the instances already serving users.

Many of the company's customers are enterprises whose networks cache DNS records for long periods, so the application's DNS name must not change.

Which deployment approach should the company use?

A Rolling

B Rolling with additional batch

C Blue/green, cloning the environment and swapping environment URLs

D Immutable

◆ PAUSE AND DECIDE

Which deployment approach should the company use?

A Rolling

Reduces capacity, and a failure leaves old instances already modified.

B Rolling with additional batch

Full capacity, but existing instances are still updated in place. A failure touches them.

C Blue/green URL swap

Clean and fast, but a URL swap is a DNS change, which long DNS caching defeats.

D Immutable

New instances in a temporary group; original instances untouched; same DNS name.

THE SIGNAL

Full capacity, untouched originals, and no DNS change: immutable. DNS caching rules out the URL swap.

A production Elastic Beanstalk environment runs on an Amazon Linux 2 platform branch, which has been retired. The team must move it to an Amazon Linux 2023 platform branch.

The move must have near-zero downtime, allow testing of the new platform before users reach it, and allow a fast return to the old platform. The database runs outside the environment.

What should the team do?

A Clone the environment onto the new platform branch, test it, then swap environment URLs

B Run an immutable deployment of the current application version to the existing environment

C Enable managed platform updates and wait for the next maintenance window

D Change the platform branch in place during a quiet period using All at once

◆ PAUSE AND DECIDE

What should the team do?

A Clone, test, swap URLs

The new platform is fully testable first, and swapping back is the fast return. The external database makes it safe.

B Immutable deployment

Deploys an application version. It does not move the environment to a new platform branch.

C Managed platform updates

Patches within the current branch. It will not cross to a different branch.

D In place, all at once

Downtime, no testing beforehand, and no quick way back.

THE SIGNAL

Platform branch change with testing and fast rollback: clone and swap. This is where the URL swap earns its place.

> An application runs on Amazon EKS. Each pod loads a large cache for about 30 seconds after starting before it can serve requests.
>
> During every rolling update, users receive errors for roughly 30 seconds per batch of new pods. Pods are never restarted unexpectedly.

What should the DevOps engineer configure?

A. A liveness probe on the container

B. A readiness probe that succeeds only once the cache has loaded

C. A larger maxSurge on the Deployment

D. A higher minimum replica count on the Horizontal Pod Autoscaler

◆ PAUSE AND DECIDE

What should the DevOps engineer configure?

A Liveness probe

Restarts stuck containers. A warming pod is not stuck; it would still get traffic too early.

B Readiness probe after cache load

The pod receives traffic only once it can serve, and the rollout waits for readiness.

C Larger maxSurge

More new pods at once, all receiving traffic before they are warm.

D More minimum replicas

More pods overall; new ones still take traffic too early.

THE SIGNAL

Traffic reaches pods before they can serve: readiness. Containers hang and need restarting: liveness.

Module 7 in five lines

- ECS rolling: min and max percent. Circuit breaker catches tasks that fail; alarms catch tasks that misbehave.
- ECS native blue/green, linear and canary since 2025: test listener, hooks, bake, instant rollback.
- Lambda: an alias splits traffic by weight. SAM's AutoPublishAlias and DeploymentPreference wire CodeDeploy.
- Beanstalk: immutable for clean releases; clone and URL swap for platform changes.
- Hooks must report back. EKS readiness probes gate traffic; liveness probes restart.

> ON THE EXAM
>
> Next, module 8: cross-account and multi-Region pipelines, the last module of Domain 1.